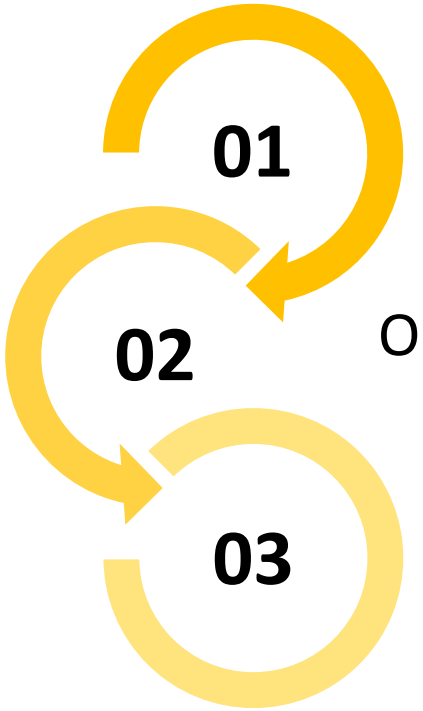

Яндекс

Яндекс Лицей

Как не перепутать апельсины с яблочным пирогом

и посчитать его площадь

Что впереди



Типы данных

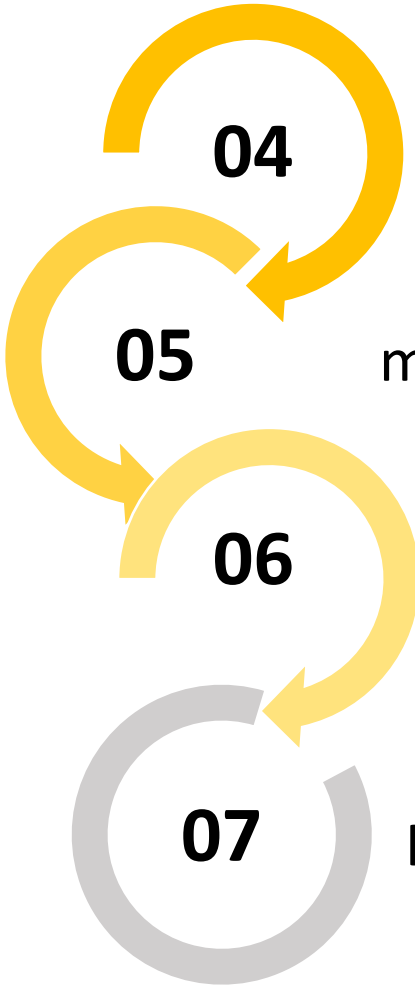
Цифры, буквы и всё, что между ними

Операции над числами

Калькулятор на минималках

Операции над строками

Строчки из кусочков



Форматирование строк

F-строки

min, max

Кто тут крайний?

Модуль math

Суперспособности для математики

Итоги



Типы данных:

Цифры, буквы и всё, что между ними, или

Почему 5 ≠ '5'

Типы данных: что известно?

Python как конструктор: каждый блок (данные) имеет свою форму (тип).

- строки **обязательно** должны быть в *кавычках*;
- input() всегда возвращает строку, пусть даже и пустую (когда пользователь просто нажал Enter).



mood = 'Хочу арбуз!'

Но что, если попробовать сложить строку и число?

Пока будет катастрофа:

```
a = input()      # Пользователь вводит '5'
print(a + 1)     # Ошибка: нельзя сложить строку '5' и число 1
```

Python: «Эй, ты пытаешься смешать текст и число? Это как добавлять ананас в пиццу — против правил!»

Целые числа

Для хранения целых чисел в Python есть специальный тип **int** (от английского **integer** — целое число).



Сказать интерпретатору, что мы хотим записать в переменную целое число, очень просто — надо его записать без всяких дополнительных уловок:

```
watermellons_count = 200  # Это целое число
```

А ещё числа в Python не имеют ограничений по размеру (в отличие от многих других языков программирования). Хотите число с миллионом цифр? Пожалуйста!

```
big_number = 12345678901234567890123456789O  # Тут не миллион цифр, но мы устали печатать
print(big_number)  # Python просто пожмёт плечами и выведет.

print(42)  # И никаких кавычек
print(1, 2, 3, 4)  # Числа будут в одной строке через пробел

desired_salary = 999_999_999_999  # Это тоже целое число
```

Целые числа

Как строку из input() (или любую другую подходящую) превратить в число? А вот так:

```
five_as_string = '5'  # Была строка
five_as_int = int(five_as_string)  # Стало число

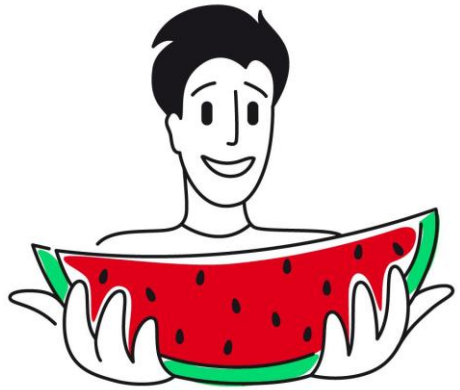
# Переводим значение, которое ввёл пользователь
print('Введите расчётное количество звёзд на небе')
stars_count = int(input())
```



Для выполнения функции необходимо вычислить все значения, записанные у неё в скобках. Поэтому прежде, чем перевести в число, тут сначала выполняется input(), который получает строку пользовательского ввода. Полученное значение бережно передаётся уже в int() для того, чтобы превратиться в целое число.

Дробные числа

Запись в переменную



- Для их хранения в Python используется тип float.

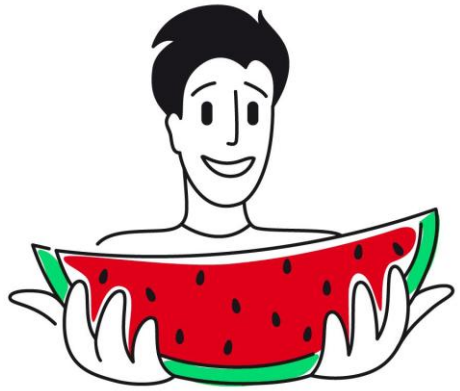
```
price = 5.0  # Желаемая средняя оценка за экзамены
temperature = -15.5  # Скоро зима
pi = 3.14  # На случай, если будем считать площадь круга
```

- Поддерживается *экспоненциальная* запись — это когда число представляется как какая-то его часть (**мантисса**), умноженная на десять в нужной степени (**порядок**).

```
avogadro = 6e23
# 600000000000000000000000 (число Авогадро с некоторым округлением)
electron_mass = 9.1e-31
# 0.000000000000000000000000000000091 кг (легче пёрышка)
```

Дробные числа

Преобразование из строки и числа



- Если вы думаете, что для получения дробного числа, например, из строки необ воспользоваться функцией float() (по аналогии с int()), то вы абсолютно правы:

```
pi = float('3.14')  # '3.14' → 3.14
float_from_int = float(3)  # 3 → 3.0
float_from_input = float(input())
# Не забываем, что разделитель — точка, а не запятая, иначе  ValueError
```

- Есть ещё специальные зарезервированные слова, из которых может получиться значение типа float:

```
inf = float('+infinity')  # Положительная бесконечность, можно получить и из '+inf'
negative_inf = float('-infinity') # Аналогично '-inf' (как желание вставать с утра по будильнику)
nan = float('nan')  # Not a number — не число, а что-то непонятное
```

Магия и странности округления

- Если мы хотим получить целое число из дробного, то можно просто передать его в функцию int() — и дробная часть отбросится.

```
price = 999.99
price_in_my_head = int(price)  # Маркетинговая уловка
```

- Но почти всегда нам хочется не отбросить дробную часть, а выполнить какое-нибудь округление, и тут на помощь придёт функция round(), которая на самом деле умеет даже немного больше, чем может показаться на первый взгляд.

```
print(round(36.6))  # О, нет, я заболел и не могу никуда идти
print(round(5.1 * 5.1))  # Площадь квадрата где-то 25 квадратных метров

# А теперь внимание — фокус
print(round(3.6))  # Ожидаемо 4
print(round(3.5))  # Конечно же, 4
print(round(2.6))  # Сто процентов 3
print(round(2.5))  # ШТА?
```

Магия и странности округления

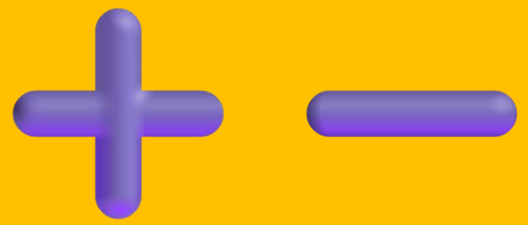
В таких пограничных случаях округление в Python происходит *к ближайшему чётному*. И это не баг, а фича!

Почему так? Такое округление называется *банковским*, оно позволяет уменьшать ошибки при множественных округлениях:

- Финансы: Если всегда округлять 0.5 вверх, банки соорудят миллионы за счёт клиентов.
- Наука: 1000 округлений вверх дадут +500 к погрешности. Банковский метод сводит её к нулю.
- Так округлять договорились в международном стандарте для чисел с плавающей точкой, а float как раз такой тип чисел.
- Ну и чётные числа прикольнее, в конце концов!

У функции round() есть второй необязательный параметр, который говорит о том, до какого знака нужно округлить число. Вот как его можно использовать:

```
print(round(3.14, 1))  # 3.1
print(round(3.01, 5))  # 3.01 — нечего округлять, у нас и так лучше точность
# Можно использовать и для округления целой части
print(round(999, -3)) # 1000 — нас не проведёшь хитрыми трюками с ценой
```



Операции над числами, или

Калькулятор на минималках

Операции с числами

Как и любой уважающий себя калькулятор, Python умеет делать следующие операции с целыми и дробными числами:

- сложение +
- вычитание -
- умножение *
- деление /
- возведение в степень (в том числе дробную,
  т. е. взятие корня нужной степени) **
- получение значения по модулю:
  функция abs()

```
int_num1 = 10
int_num2 = 3
float_num1 = 2.5
float_num2 = 1.5

print(int_num1 + int_num2)  # 13
print(int_num1 + float_num1)  # 12.5
print(float_num2 + float_num1)  # 4.0 — если хоть один операнд float — результат float

print(int_num1 - int_num2)  # 7
print(int_num1 * int_num2)  # 30
print(int_num1 / int_num2)  # 3.333... деление всегда возвращает float
print(int_num1 ** int_num2)  # 1000 — возведение в степень
print(4 ** 0.5)  # 2 — квадратный корень

abs_num = abs(-1)
print(abs_num, abs(-3.99))
```

Целочисленное деление



Делите пиццу на друзей: // — сколько кусков каждый съест, % — сколько останется в коробке.

Целочисленное деление Остаток от деления

```
pirates = 10
gold_coins = 103

coins_per_pirate = gold_coins // pirates  # 10 (каждый получает по 10)
leftover_coins = gold_coins % pirates     # 3 (остаток)
```

Не пренебрегайте операциями целочисленного деления и получения остатка — часто с их помощью можно сделать алгоритмы значительно элегантнее, чем без них. Например, самая простая **проверка на чётность** — это получить *остаток от деления на 2*.

Если надо получить сразу и результат целочисленного деления, и остаток. В таких случаях на помощь придёт функция divmod():

```
coins_per_pirate, leftover_coins = divmod(gold_coins, pirates)
```

Приоритет операций

Тут без сюрпризов, всё как в математике:

- Скобки ()
- Степень **
- Умножение *, деление /, целочисленное деление //, остаток %
- Сложение +, вычитание -

```
print(2 + 3 * 4)   # 14 (сначала умножение)
print((2 + 3) * 4)  # 20 (скобки рулят)
```



Операции над строками —

строчки из кусочков

Операции над строками

- Строку можно склеить с другой строкой с помощью обычного оператора +, который мы использовали с числами. Эта операция называется **конкатенацией**.

```
boring_days = 'пн, вт, ср, чт, пт'
good_days = 'сб, вс'
week = boring_days + ', ' + good_days
print(week)
```

Осторожно:

'5' + 5 вызовет ошибку.

Python не умеет складывать строки с числами (и правильно делает).

- Мы можем **дублировать** строку нужное количество раз, умножив её на **целое** число (или наоборот число повторений умножить на строку).
- Мы можем узнать, сколько символов в строке (то есть её длину), с помощью функции len():
- А ещё мы можем превратить любое целое или вещественное число в строку с помощью функции str():

```
laugh = 'ха' * 3  # 'хахаха'
border = 40 * '-'  # '----------'
```

```
text = 'Строка'
length = len(text)  # 6 (символов)
print(len(''))  # Длина пустой строки — 0 символов
```

```
num1 = 3
num2 = 10
my_str = str(num1 * num2) * num1
# Сначала получаем число 30, потом строку '30'
и повторяем её 3 раза -> '303030'
```

Классно, что + работает и для чисел, и для строк, а * не только для чисел, но и для пары из целого числа и строки. Умение одного и того же кода работать с разными типами данных называется **полиморфизмом.**

f-строки — держите его, а то он тут всё отформатирует



f-строки (*formatted strings*) — это как волшебные чернила: вписываешь переменные в текст, и они появляются сами! А всего-то надо добавить f перед открывающей кавычкой и вписать переменные в нужные места внутри фигурных скобок:

```
name = 'Вася'
age = 15
print(f'Меня зовут {name}, и мне {age} лет.')
# Меня зовут Вася, и мне 15 лет.
```

В f-строках можно проводить вычисления:

```
num1 = 2
num2 = 2
print(f"{num1} + {num2} = {num1 + num2}")
# 2 + 2 = 4 — гениально, не правда ли?
```

Всё это форматирование выглядит немного пугающе (особенно для второго урока), но его не обязательно помнить наизусть (мало кто помнит) — достаточно знать, что такое есть, и пользоваться сайтами вроде https://pyformat.info/ при необходимости.

А также форматировать числа перед выводом:

```
pi = 3.1415926535
hours = 2
minutes = 9
print(f"π ≈ {pi:.2f}")      # π ≈ 3.14 (два знака после запятой)
print(f"{hours:02d}:{minutes:02d}") # 02:09 — добавляем ведущий ноль
print(f"Двоичное: {42:b}")   # Двоичное: 101010
print(f"Процент: {0.25:%}")  # Процент: 25.000000%
```

min, max — кто тут крайний?



Функции min() и max() нужны для нахождения соответственно минимума и максимума из тех значений, что вы в них передали.

- Можно работать с целыми и вещественными числами:

```
print(max(1, 2.8, 3, 4.1, 5))  # 5 — царь горы
print(min(-5, 10, 3.2))  # -5 — лузер месяца
```

- Со строками. Сравнение идет лексикографически — чем дальше слово стояло бы в словаре, тем оно «больше», но все большие буквы идут перед всеми маленькими.

```
print(min("Иван", "Анна", "Ян"))  # Анна (сравнивает по первой букве)
print(max("Анна", "анна")) # анна — все большие буквы идут раньше маленьких
```

- С одной строкой, тогда min() или max() интерпретируют её как коллекцию букв и вернёт лексикографический минимум или максимум этой коллекции.

```
max_letter = max('Одна строка')
print(max_letter)  # 'т' — дальше всего от начала нашего воображаемого словаря
```

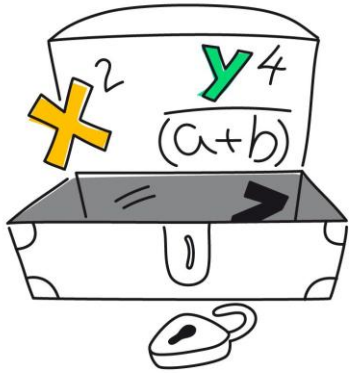
Если мы попробуем посчитать максимум или минимум от одного числа или смешаем вместе числа и строки, то ничего хорошего не выйдет и наша программа завершится с ошибкой.



Модуль math — суперспособности для математики

Модуль math

Модуль, или *библиотека*, — это набор функций, которые обычно объединены какой-либо предметной областью. В Python есть много встроенных библиотек, с некоторыми из них мы со временем познакомимся.



Библиотека math — это как чемоданчик шпиона-математика: внутри сотни функций для сложных вычислений. Но сначала его нужно «открыть» — то есть импортировать.

```
import math
```

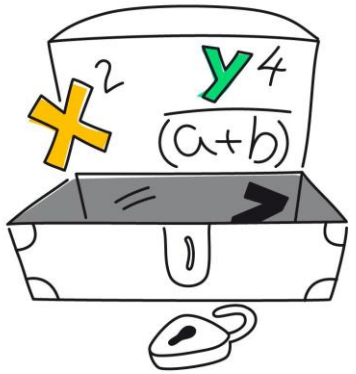
```
import math

print(math.pi)  # 3.141592653589793
```

Чтобы обратиться к каким-нибудь возможностям подключённой библиотеки, надо написать её имя, потом точку, а потом имя желаемой функции или переменной. Например, так можно получить значение числа пи:

Для импорта используется команда import (как же здорово, когда всё логично). Импорты нужных модулей по **PEP8** пишутся в самом начале перед всем остальным кодом и отделяются от него одной пустой строкой.

Модуль math



Давайте посмотрим на несколько функций из модуля math.

- **math.sqrt():** Квадратный корень.

```
print(math.sqrt(25))  # 5.0
```

- **math.ceil():** Округление вверх.

```
print(math.ceil(2.3))  # 3 (даже 2.1 станет 3)
```

- **math.floor():** Округление вниз.

```
print(math.floor(2.9))  # 2 (даже 2.999 не станет 3)
```

- **math.hypot():** Значение гипотенузы по двум катетам

```
a, b = 3, 4
c = math.hypot(a, b)  # 5.0 (теорема Пифагора)
```

- **math.log():** Логарифм числа по произвольному основанию, то есть в какую степень надо возвести основание, чтобы получилось заданное число.

```
print(math.log(100, 10))  # 2.0 → потому что 10² = 100
print(math.log(8, 2))     # 3.0 → 2³ = 8
# Если не указать второй аргумент, то будет по основанию e
print(math.log(math.e)) # 1.0, потому что e и есть e в первой степени
# у функции есть ещё несколько функций-братьев
print(math.log10(100)) # По основанию 10
print(math.log2(8)) # По основанию 2
```

Можно много говорить о том, какие логарифмы крутые и как невероятно упростили расчёты, например, астрономам, когда не было не только калькулятора, но и даже кофе в Европу ещё не завезли.

Делим пиццу на π кусков



В math содержатся и тригонометрические функции для вычисления синуса, косинуса и т. д.

Математики XVIII века решили: «Хватит это терпеть! Давайте привяжем углы к радиусу». Поэтому:

- 1 радиан — это угол, при котором длина дуги равна радиусу.
- 1 полный оборот = 2π радиан (~6.28). Почему? Потому что длина окружности равна 2πr.

Формулы для перевода из градусов в радианы и обратно:

- радианы = градусы × (π / 180)
- градусы = радианы × (180 / π)

```
angle_deg = 180
angle_rad = math.radians(angle_deg)  # Ровно π радиан
angle_deg = math.degrees(angle_rad)  # Снова 180 градусов
```

```
angle_deg = 45
angle_rad = math.radians(angle_deg)  # Конвертация в радианы

print(math.sin(angle_rad))     # ≈ 0.7071
print(math.cos(angle_rad))     # ≈ 0.7071
print(math.tan(angle_rad))     # ≈ 1.0
print(math.degrees(math.asin(0.7071)))
# Из значения синуса в значение угла в радианах и обратно в градусы ≈ 45.0
```

Если подсунуть градусы — получите баги, достойные античной трагедии.

Итог: Математика побеждена! (Но это не точно.)



Вы узнали:

- чем числа отличаются от строк и наоборот;
- как поделить поровну что угодно;
- как два раза не повторять ~~два раза не повторять~~ одно и то же;
- как импортировать библиотеки;
- почему в ГДЗ по математике нет никакого смысла (как будто до этого урока он был).

Несмотря на все старания, мы не рассмотрели даже половины функций модуля math. Если вам хочется узнать, что же там есть ещё, то наберитесь смелости и загляните в документацию.

P. S. В любой непонятной ситуации глубоко вдохните, вспомните, что нет ничего невозможного, и округляйте до ближайшего чётного.